Ejercicio Demostrativo

Heaps ordenamiento y búsqueda — Unidad 5

Auxiliar: Sergio Sebastian Sandoval Ruiz

Tiempo estimado: 60 min

5 minutos para Tolerancia y 55 minutos de desarrollo académico

Universidad de San Carlos de Guatemala
Facultad de Ingeniería
Escuela de Ingeniería en Ciencias y Sistemas
Segundo Semestre 2026
Laboratorio de Estructuras de Datos C

1. Marco Formativo

1.1 Valores

Tolerancia	Aceptar las diferencias sin discriminar. Al comparar estrategias de ordenamiento, escuchar las propuestas y respetar los ritmos de aprendizaje. Comprobar cada solución con datos y argumentos sin descalificar a quien eligió otro procedimiento.

1.2 Competencias

Tipo de competencia	Redacción
Competencia general	Implementa estructuras de datos lineales y no lineales mediante programación orientada a objetos para desarrollar soluciones eficientes.
Competencia específica	Implementa un max-heap en un arreglo, ejecuta inserción y extracción, compara sus reglas con un min-heap y aplica ordenamiento por heapsort e inserción, así como búsqueda secuencial y binaria.

2. Palabras Clave

Heap binario: Árbol completo con una relación de prioridad entre padres e hijos.

Max-heap: Cada padre es mayor o igual que sus hijos; el máximo está en la raíz.

Min-heap: Cada padre es menor o igual que sus hijos; el mínimo está en la raíz.

Ascenso: Ajuste de un elemento hacia su padre después de insertar.

Descenso: Ajuste de un elemento hacia un hijo después de reemplazar la raíz.

Heapsort: Ordenamiento que construye un heap y fija extremos sucesivamente.

Inserción: Ordenamiento que amplía un prefijo ya ordenado.

Búsqueda binaria: Consulta que reduce a la mitad un intervalo de un arreglo ordenado.

3. Resumen

La demostración de la unidad 5 desarrolla los cinco subtemas de la planificación: definición de Heaps, tipos de Heaps, algoritmos de ordenamiento, algoritmos de búsqueda y aplicación de algoritmos. El programa permite observar un max-heap y comprobar la diferencia entre conservar prioridad y ordenar todo el arreglo.

El auxiliar presenta una traza antes de ejecutar el programa y contrasta la salida con resultados calculados a mano. El estudiante debe explicar los cambios de estado y justificar las condiciones de cada operación.

4. Contenido

4.1 DESCRIPCIÓN DEL EJERCICIO

Construir un max-heap con `[20,15,18,8,10,12]`, insertar 25 y extraer la raíz. Luego ordenar copias de `[9,4,7,1,3]` mediante heapsort e inserción. Buscar 7 en el arreglo ordenado y verificar la ausencia de 6 con búsqueda binaria.

4.2 OBJETIVO DE LA DEMOSTRACIÓN

Demostrar la representación de un heap completo mediante índices, restaurar su propiedad tras cada cambio y aplicar algoritmos de ordenamiento y búsqueda con sus precondiciones. Explicar que un heap no ofrece orden total ni habilita por sí solo una búsqueda binaria.

4.3 RECURSOS NECESARIOS

- Java JDK 11 o superior, editor o IDE y terminal con javac y java.
- Dos archivos en el mismo directorio: `MaxHeap.java` y `Main.java`.
- Diapositivas de Heaps ordenamiento y búsqueda de la unidad 5.
- Pizarrón o papel para trazar índices, estados y resultados.

Distribución del tiempo

Actividad	Minutos
Tolerancia y participación	5
Conceptos y traza inicial	15
Implementación en Java	20
Pruebas en consola	10
Contraste y cálculo manual	5

Actividad	Minutos
Resultados y cierre	5

4.4 DESARROLLO PASO A PASO

Con índices base cero, la raíz está en 0, los hijos de i están en 2i+1 y 2i+2, y el padre de i>0 está en (i-1)/2 con división entera. El tamaño activo indica qué posiciones pertenecen al heap. Las claves repetidas son válidas.

Paso 1 — Valor y participación (5 min)

Aceptar las diferencias sin discriminar. Al comparar estrategias de ordenamiento, escuchar las propuestas y respetar los ritmos de aprendizaje. Comprobar cada solución con datos y argumentos sin descalificar a quien eligió otro procedimiento.

Solicitar una propuesta antes de mostrar la respuesta. Comparar los procedimientos con la misma entrada y cuidar que la revisión permita comprender el error y corregirlo.

Paso 2 — Conceptos y traza (15 min)

Dibujar el árbol y verificar cada relación padre-hijo. Explicar el caso min-heap invirtiendo la comparación: el menor queda en la raíz. Elegir el tipo según el significado de prioridad, no según el orden en que se recibieron los datos.

Operación	Arreglo activo	Tamaño
Construir	`[20,15,18,8,10,12]`	6
Insertar 25 y ascender	`[25,15,20,8,10,12,18]`	7
Extraer 25 y descender	`[20,15,18,8,10,12]`	6

Qué observar:

- Los nodos se guardan por niveles y no se dejan huecos en el segmento activo.
- El padre domina a sus hijos, pero los hermanos no tienen que estar ordenados.

- El heap admite hasta 20 elementos. Insertar lleno o extraer vacío produce una excepción.
- Para pasar conceptualmente a min-heap se invierte la prioridad y se elige el hijo menor durante el descenso.

Paso 3 — Implementación (20 min)

Crear `MaxHeap.java`. El archivo contiene la clase pública MaxHeap y una clase auxiliar Algoritmos. Mantener ambas en el mismo archivo. La primera parte implementa las operaciones del heap y el descenso compartido por heapsort.

java

```java
public class MaxHeap {
    private final int[] datos = new int[20];
    private int n;
    public void insertar(int x) {
        if (n == datos.length)
            throw new IllegalStateException("Heap lleno");
        int i = n++; datos[i] = x;
        while (i > 0) {
            int p = (i-1)/2;
            if (datos[p] >= datos[i]) break;
            int t = datos[p]; datos[p] = datos[i]; datos[i] = t;
            i = p;
        }
    }
    public int extraer() {
        if (n == 0) throw new IllegalStateException("Heap vacio");
        int maximo = datos[0]; datos[0] = datos[--n];
        Algoritmos.descender(datos,0,n);
        return maximo;
    }
    public int maximo() {
        if (n == 0) throw new IllegalStateException("Heap vacio");
        return datos[0];
```

```
    }
    public int size() { return n; }
    public void mostrar() {
        for (int i = 0; i < n; i++) System.out.print(datos[i]+" ");
        System.out.println();
    }
}
class Algoritmos {
    static void descender(int[] a, int i, int n) {
        while (2*i+1 < n) {
            int h = 2*i+1;
            if (h+1 < n && a[h+1] > a[h]) h++;
            if (a[i] >= a[h]) break;
            int t = a[i]; a[i] = a[h]; a[h] = t;
            i = h;
        }
    }
```

Continuación de la implementación:

La clase Algoritmos continúa en la segunda parte. Heapsort usa un max-heap para producir orden ascendente. En el descenso, n es un límite exclusivo: el sufijo ya ordenado no pertenece al heap activo.

java

```
    static void heapsort(int[] a) {
        for (int i = a.length/2-1; i >= 0; i--)
            descender(a,i,a.length);
        for (int fin = a.length-1; fin > 0; fin--) {
            int t = a[0]; a[0] = a[fin]; a[fin] = t;
            descender(a,0,fin);
        }
    }
    static void insercion(int[] a) {
```

```
        for (int i = 1; i < a.length; i++) {
            int x = a[i], j = i-1;
            while (j >= 0 && a[j] > x) { a[j+1] = a[j]; j--; }
            a[j+1] = x;
        }
    }
    static int lineal(int[] a, int x) {
        for (int i = 0; i < a.length; i++)
            if (a[i] == x) return i;
        return -1;
    }
    static int binaria(int[] a, int x) {
        int izq = 0, der = a.length-1;
        while (izq <= der) {
            int m = izq+(der-izq)/2;
            if (a[m] == x) return m;
            if (a[m] < x) izq = m+1;
            else der = m-1;
        }
        return -1;
    }
}
```

Copiar las dos partes consecutivamente en el archivo indicado. No agregar una segunda declaración de clase ni omitir las llaves de cierre. La prueba del siguiente paso utiliza esta misma implementación.

Paso 4 — Pruebas en consola (10 min)

Crear `Main.java` en el mismo directorio. La función clone copia el arreglo de entrada para que ambos ordenamientos reciban los mismos datos; no realiza el ordenamiento. El programa no utiliza una cola de prioridad ni un método de ordenamiento de biblioteca.

java

```
public class Main {
    public static void main(String[] args) {
```

```
        MaxHeap h = new MaxHeap();
        for (int v : new int[]{20,15,18,8,10,12}) h.insertar(v);
        System.out.print("Heap: "); h.mostrar();
        h.insertar(25);
        System.out.print("Insertar 25: "); h.mostrar();
        System.out.println("Extraido: "+h.extraer());
        System.out.print("Heap final: "); h.mostrar();
        int[] a = {9,4,7,1,3}, b = a.clone();
        Algoritmos.heapsort(a); Algoritmos.insercion(b);
        System.out.print("Heapsort:");
        for (int v : a) System.out.print(" "+v);
        System.out.print("\nInsercion:");
        for (int v : b) System.out.print(" "+v);
        System.out.println();
        System.out.println("Lineal 7: "+Algoritmos.lineal(a,7));
        System.out.println("Binaria 7: "+Algoritmos.binaria(a,7));
        System.out.println("Binaria 6: "+Algoritmos.binaria(a,6));
    }
}
```

Compilar y ejecutar:

```
javac MaxHeap.java Main.java
java Main
```

Salida esperada:

```
Heap: 20 15 18 8 10 12
Insertar 25: 25 15 20 8 10 12 18
Extraido: 25
Heap final: 20 15 18 8 10 12
Heapsort: 1 3 4 7 9
Insercion: 1 3 4 7 9
Lineal 7: 3
```

```
Binaria 7: 3
Binaria 6: -1
```

Los índices devueltos corresponden al arreglo ordenado. El número 7 ocupa la posición 3, contando desde cero. La ausencia se informa con -1. Ese marcador es un índice inválido, no una restricción sobre los valores que pueden almacenarse.

Paso 5 — Trazas de ordenamiento y búsqueda (5 min)

En inserción, `[9,4,7,1,3]` pasa por `[4,9,7,1,3]`, `[4,7,9,1,3]`, `[1,4,7,9,3]` y `[1,3,4,7,9]`. Solo se desplazan valores estrictamente mayores, por lo que la versión respeta el orden relativo de claves iguales.

En heapsort, cada intercambio de la raíz con el último elemento activo fija el máximo restante. El descenso restaura únicamente el prefijo pendiente. La versión implementada utiliza memoria auxiliar constante y no es estable en general.

En la búsqueda binaria de 7, el centro inicial de `[1,3,4,7,9]` es el índice 2, con valor 4. Se conserva el intervalo `[3,4]`; su nuevo centro es 3 y encuentra 7. Para buscar 6 se continúa hasta que el intervalo queda vacío.

Situación	Elección y justificación
Atender mayor prioridad	Max-heap; raíz accesible en O(1)
Pocos datos casi ordenados	Inserción puede evitar desplazamientos
Ordenar sin arreglo auxiliar	Heapsort iterativo
Buscar repetidamente por clave	Ordenar y utilizar búsqueda binaria

Pedir una explicación del resultado, no únicamente su lectura. El cálculo manual debe utilizar exactamente los datos y el orden indicados. Si cambia una entrada, se deben actualizar también las expectativas de la prueba.

Observaciones Importantes

- No buscar con búsqueda binaria directamente sobre el arreglo de un heap.
- Al extraer la única clave, el tamaño pasa a cero y el descenso no accede a un hijo inexistente.

- El criterio de comparación debe ser coherente. Un empate de prioridades no garantiza orden de llegada en el heap.
- Contar el costo de ordenar si la aplicación solo hará una consulta. Una búsqueda lineal puede ser suficiente.

4.5 Tabla Comparativa

TABLA COMPARATIVA

Operación	Tiempo	Observación
Consultar máximo	O(1)	Heap no vacío
Insertar o extraer	O(log n) peor caso	Capacidad disponible
Heapsort	O(n log n) peor caso	Memoria auxiliar O(1)
Inserción	$O(n^2)$ peor caso	O(n) si ya está ordenado
Búsqueda secuencial	O(n) peor caso	Sin requisito de orden
Búsqueda binaria	O(log n) peor caso	Arreglo ordenado

4.6 Resultado Esperado

El estudiante debe justificar los intercambios al insertar y extraer, obtener el mismo arreglo ordenado con ambos métodos y explicar cada cambio de límites en búsqueda binaria. Los algoritmos se prueban también con arreglos vacíos, repetidos y valores negativos.

Verificación	Resultado
Propiedad tras insertar 25	25 queda en la raíz
Extracción	Se retira 25 y se conserva el heap
Orden ascendente	`[1,3,4,7,9]` en ambos métodos

Verificación	Resultado
Consulta y ausencia	7 devuelve 3; 6 devuelve -1

4.7 Aplicación Práctica

Un heap sirve para seleccionar la siguiente tarea por prioridad sin ordenar toda la colección después de cada inserción. Los ordenamientos permiten preparar catálogos y reportes. La búsqueda binaria es útil en arreglos que permanecen ordenados entre consultas; las nuevas inserciones deben conservar ese orden.

Paso 6 — Resultados y cierre (5 min)

Comparar en parejas dos procedimientos para ordenar la misma entrada. Cada integrante escucha el razonamiento del otro y revisa la corrección con una traza. Cerrar preguntando por qué el heap conserva prioridad, pero no un orden completo.

5. Recomendaciones al siguiente tutor

Dificultades previstas:

- Los errores previsibles son usar el índice de hijo de un esquema base uno, incluir el sufijo ordenado en el descenso y dejar el mismo punto medio dentro del siguiente intervalo de búsqueda. Conviene dibujar el tamaño activo antes de cambiarlo.

Recomendaciones para futuras sesiones:

- Preparar la secuencia de prueba y señalar los índices debajo de cada valor.
- Comparar min-heap y max-heap con las mismas claves.
- Mantener una copia independiente para cada ordenamiento y conservar los duplicados.
- Pedir al estudiante que justifique las precondiciones antes de ejecutar la búsqueda binaria.

6. Referencias

1. Sedgewick, R., y Wayne, K. Algorithms, 4th edition. Priority Queues. https://algs4.cs.princeton.edu/24pq/
2. Princeton University. Sorting and Searching. https://introcs.cs.princeton.edu/python/42sort/
3. Material de clase. Heaps ordenamiento y búsqueda, unidad 5. Laboratorio de Estructuras de Datos C, Escuela de Ingeniería en Ciencias y Sistemas, USAC, 2026.